PROJETO INTEGRADO · DESENVOLVIMENTO MÓVEL · PUC MINAS · 2026/2

SPRINT 3 ARMAZENAMENTO E PERSISTÊNCIA

Projeto do banco de dados SQLite do TripWise e integração com operações CRUD no app Flutter.



PASSAGEIROS

Ana Luiza Ribas
Ana Paula Lopes
Débora Lino
Thiago Bahia

BANCO

SQLite (sqflite) · 10 tabelas

REPOSITÓRIO

github.com/tbahiasantos/tripwise

GATE

S3

ENTREGA

26/10/2026

REQUISITOS

O QUE ESTA SPRINT ENTREGA

Itens pedidos na Sprint 3 e onde cada um aparece neste documento.

	REQUISITO	COMO FOI ATENDIDO	
✓	**Projeto do banco de dados**	Modelo entidade-relacionamento (10 tabelas), dicionário de dados, restrições (PK, FK, UNIQUE, CHECK, ON DELETE CASCADE) e script SQL de criação	p. 4 a 7
✓	**Integração do banco com o app**	Banco SQLite local com o pacote `sqflite`, acessado por repositórios a partir de uma classe `DatabaseHelper`, no mesmo padrão do SQLHelper visto em aula	p. 3 e 12
✓	**Operações CRUD**	Create, Read, Update e Delete em usuários, viagens e itens do roteiro; Create/Delete em favoritos; Create/Read em buscas	p. 8 a 11
✓	**Persistência de preferências**	Sessão do usuário logado guardada com `shared_preferences`: ao reabrir o app, ele entra direto na Home	p. 3
✓	**Demonstração em aula**	Roteiro de demonstração passo a passo e testes automatizados do CRUD em banco em memória	p. 13

BANCO LOCAL
SQLite · sqflite 2.4

PREFERÊNCIAS
shared_preferences 2.5

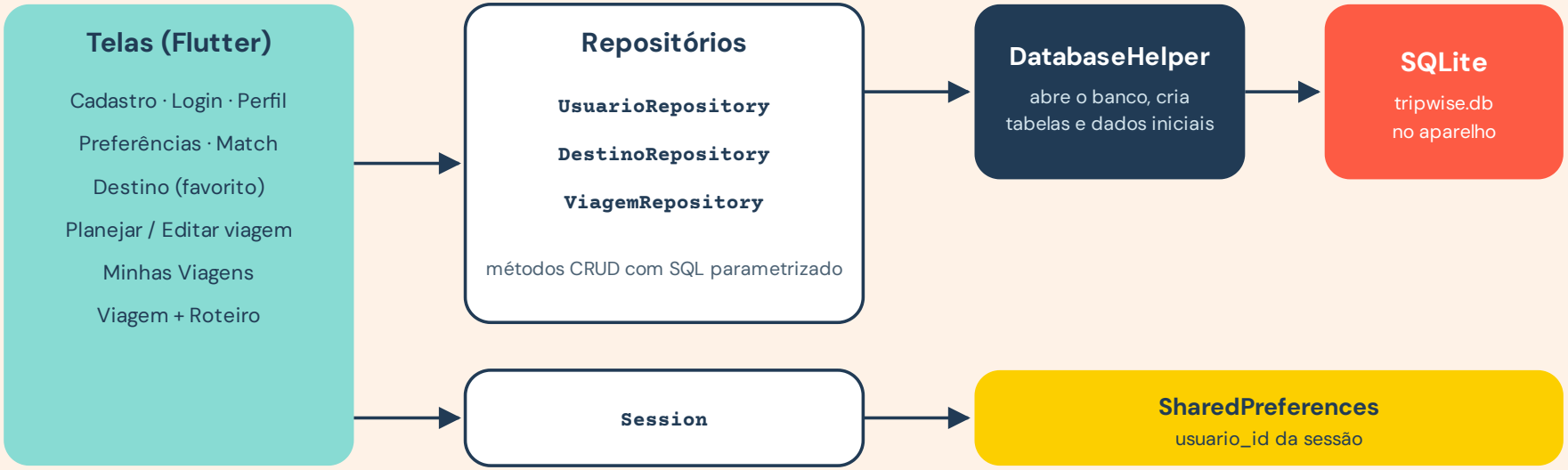
SEGURANÇA
senha com hash SHA-256 (crypto)

WEB (TESTES)
sqflite_common_ffi_web

PROJETO DO BANCO

ARQUITETURA DA PERSISTÊNCIA

Como as telas chegam até o banco. Cada tabela tem um repositório com os métodos de CRUD.



Banco local (SQLite)
Os dados ficam no próprio aparelho, sem depender de internet, como na aula de SQLite e ListView.

Normalização
Estilos, custos e roteiros em tabelas próprias. A relação N:N usuário–estilo usa a tabela associativa usuario_estilos.

Integridade
PRAGMA foreign_keys = ON, chaves estrangeiras com ON DELETE CASCADE e CHECK em status, unidade e valores.

Segurança
A senha nunca é gravada em texto puro: guardamos o hash SHA-256 de e-mail + senha.

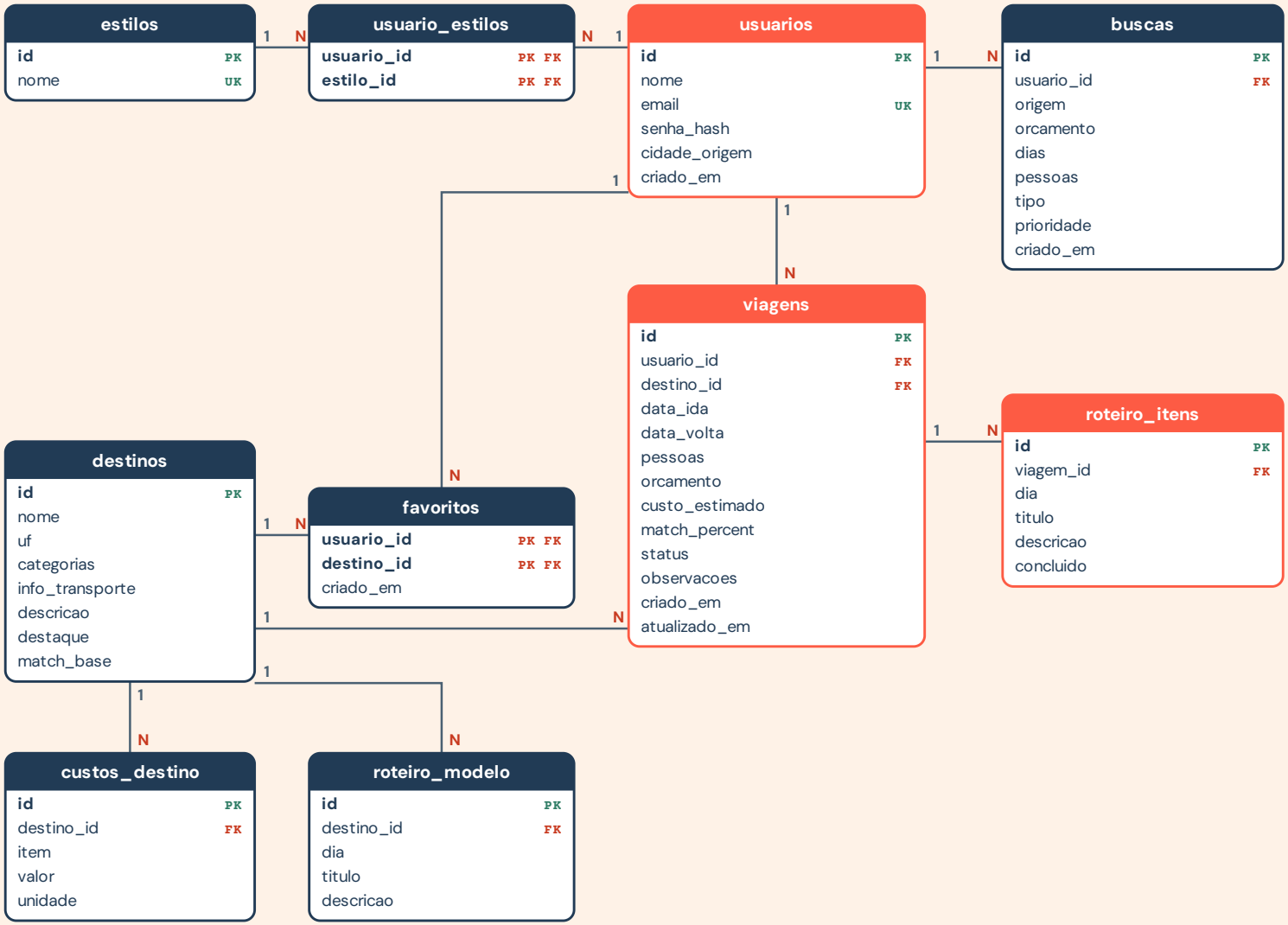
Transações
Cadastro (usuário + estilos) e criação de viagem (viagem + roteiro) gravam tudo ou nada.

Dados iniciais
Na criação do banco são inseridos 6 estilos, 8 destinos, 32 custos e 40 dias de roteiro sugerido.

PROJETO DO BANCO

MODELO ENTIDADE-RELACIONAMENTO

10 tabelas. Em vermelho, o núcleo do app: usuário → viagens → itens do roteiro. PK = chave primária, FK = chave estrangeira, UK = única.



PROJETO DO BANCO

DICIONÁRIO DE DADOS (1/2)

Usuários e catálogo de destinos.

usuarios

Usuários cadastrados no app.

id	INTEGER	PK, autoincremento
nome	TEXT	obrigatório
email	TEXT	obrigatório, único (login)
senha_hash	TEXT	SHA-256 de e-mail + senha
cidade_origem	TEXT	opcional
criado_em	TEXT	data/hora do cadastro

estilos

Catálogo de estilos de viajante.

id	INTEGER	PK
nome	TEXT	único: Aventureiro, Relax…

usuario_estilos

Associação N:N usuário × estilo.

usuario_id	INTEGER	PK, FK → usuarios (cascade)
estilo_id	INTEGER	PK, FK → estilos (cascade)

destinos

Destinos avaliados pelo Travel Match.

id	INTEGER	PK
nome / uf	TEXT	nome e estado
categorias	TEXT	ex.: "Praia,Cultura"
info_transporte	TEXT	ex.: "voo 1h50"
descricao / destaque	TEXT	textos exibidos no app
match_base	INTEGER	nota base, CHECK 0–100

custos_destino

Custos de referência por destino.

id	INTEGER	PK
destino_id	INTEGER	FK → destinos (cascade)
item	TEXT	Passagens, Hospedagem…
valor	REAL	CHECK ≥ 0
unidade	TEXT	CHECK: pessoa, noite ou pessoa_dia

roteiro_modelo

Roteiro sugerido de cada destino.

id	INTEGER	PK
destino_id	INTEGER	FK → destinos (cascade)
dia	INTEGER	dia da viagem
titulo / descricao	TEXT	atividade sugerida

PROJETO DO BANCO

DICIONÁRIO DE DADOS (2/2)

Dados gerados pelo uso do app.

viagens

Viagens planejadas pelo usuário.

id	INTEGER	PK
usuario_id	INTEGER	FK → usuarios (cascade)
destino_id	INTEGER	FK → destinos
data_ida / data_volta	TEXT	AAAA-MM-DD
pessoas	INTEGER	CHECK > 0
orcamento / custo_estimado	REAL	em R$
match_percent	INTEGER	compatibilidade na criação
status	TEXT	CHECK: Planejada, Confirmada, Concluída
observacoes	TEXT	opcional
criado_em / atualizado_em	TEXT	auditoria

roteiro_itens

Atividades do roteiro de uma viagem.

id	INTEGER	PK
viagem_id	INTEGER	FK → viagens (cascade)
dia	INTEGER	CHECK > 0
titulo	TEXT	obrigatório
descricao	TEXT	opcional
concluido	INTEGER	0 ou 1 (CHECK)

buscas

Histórico do questionário do match.

id	INTEGER	PK
usuario_id	INTEGER	FK → usuarios (cascade)
origem	TEXT	cidade de saída
orcamento	REAL	valor total em R$
dias / pessoas	INTEGER	duração e viajantes
tipo / prioridade	TEXT	Praia… / Economia…
criado_em	TEXT	data/hora

favoritos

Destinos favoritados.

usuario_id	INTEGER	PK, FK → usuarios (cascade)
destino_id	INTEGER	PK, FK → destinos (cascade)
criado_em	TEXT	data/hora

PROJETO DO BANCO

SCRIPT SQL

Trecho do script de criação (lib/data/schema.dart) e consultas usadas no app. Os 10 CREATE TABLE completos estão no repositório.

```
PRAGMA foreign_keys = ON;   -- ativado ao abrir o banco

CREATE TABLE viagens (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  usuario_id     INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  destino_id     INTEGER NOT NULL REFERENCES destinos(id),
  data_ida       TEXT    NOT NULL,
  data_volta     TEXT    NOT NULL,
  pessoas        INTEGER NOT NULL CHECK (pessoas > 0),
  orcamento      REAL    NOT NULL,
  custo_estimado REAL    NOT NULL,
  match_percent  INTEGER NOT NULL,
  status         TEXT    NOT NULL DEFAULT 'Planejada'
                 CHECK (status IN ('Planejada','Confirmada','Concluída')),
  observacoes    TEXT,
  criado_em      TEXT    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em  TEXT
);

CREATE TABLE roteiro_itens (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  viagem_id INTEGER NOT NULL REFERENCES viagens(id) ON DELETE CASCADE,
  dia       INTEGER NOT NULL CHECK (dia > 0),
  titulo    TEXT    NOT NULL,
  descricao TEXT,
  concluido INTEGER NOT NULL DEFAULT 0 CHECK (concluido IN (0,1))
);

CREATE INDEX idx_viagens_usuario ON viagens(usuario_id);
CREATE INDEX idx_roteiro_viagem  ON roteiro_itens(viagem_id, dia);
```

```
-- Lista de viagens (Minhas Viagens)
SELECT v.*, d.nome AS destino_nome, d.uf AS destino_uf
FROM viagens v JOIN destinos d ON d.id = v.destino_id
WHERE v.usuario_id = ?
ORDER BY v.data_ida;

-- Estilos do usuário (Perfil)
SELECT e.nome FROM estilos e
JOIN usuario_estilos ue ON ue.estilo_id = e.id
WHERE ue.usuario_id = ?;

-- Login
SELECT * FROM usuarios
WHERE email = ? AND senha_hash = ?
LIMIT 1;
```

Parâmetros (?)

Todas as consultas usam whereArgs, o que evita SQL injection.

Versão do banco

openDatabase(version: 1, onCreate: …). Mudanças futuras usam onUpgrade.

INTEGRAÇÃO COM O APP

MATRIZ CRUD

Onde cada operação acontece no app, por tabela.

Tabela	C · Create	R · Read	U · Update	D · Delete
usuarios	Cadastro	Login, Home, Perfil	Perfil (nome, cidade)	Perfil → Excluir conta
usuario_estilos	Cadastro	Perfil	Perfil (troca de estilos)	Cascata ao excluir conta
viagens	Planejar esta viagem	Home, Minhas Viagens, Viagem	Editar viagem	Minhas Viagens (menu/arrastar), Viagem
roteiro_itens	+ Atividade; cópia do roteiro sugerido	Tela da viagem	Editar atividade, marcar feito	Lixeira da atividade; cascata
favoritos	♥ no destino	♥ preenchido ao abrir	—	♥ de novo (desfavoritar)
buscas	Encontrar meu match	Preenche o questionário	—	Cascata ao excluir conta
destinos, custos, roteiro_modelo	Dados iniciais (onCreate)	Match, Destino, Planejar	—	—

CRUD · USUÁRIO

CADASTRO, LOGIN E PERFIL

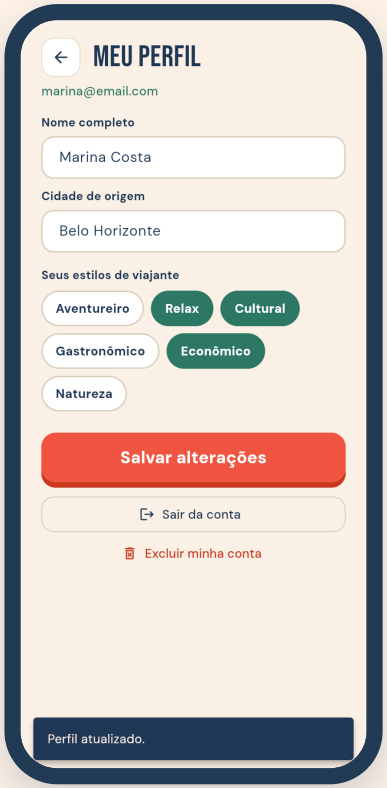
INSERT no cadastro, SELECT no login e na Home, UPDATE e DELETE no perfil. A sessão fica no SharedPreferences.



C · C · Cadastro



R · R · Home com dados do banco



U · U · Editar perfil



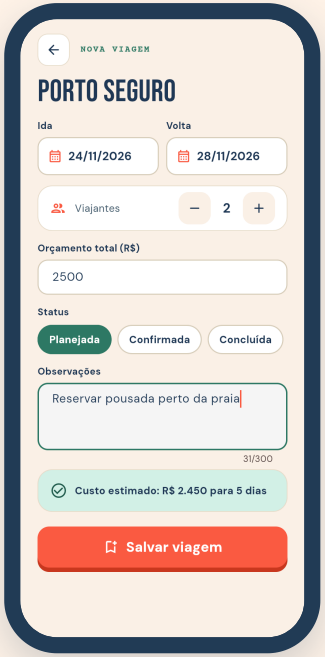
D · D · Excluir conta

PLANEJAR, LISTAR, EDITAR E EXCLUIR

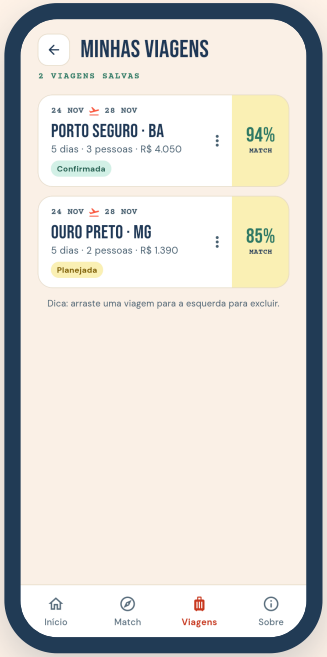
O match lê os destinos do banco. “Planejar esta viagem” grava a viagem e copia o roteiro sugerido em uma transação.



R · Match (destinos do banco)



C · Planejar viagem



R · Minhas Viagens



U · Editar viagem



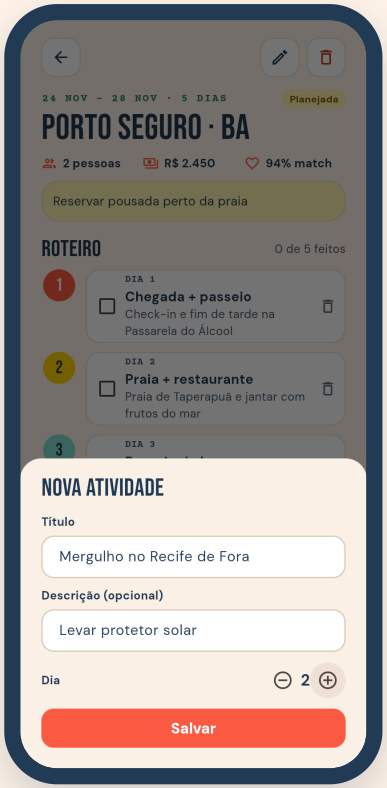
D · Excluir viagem

ATIVIDADES DO ROTEIRO

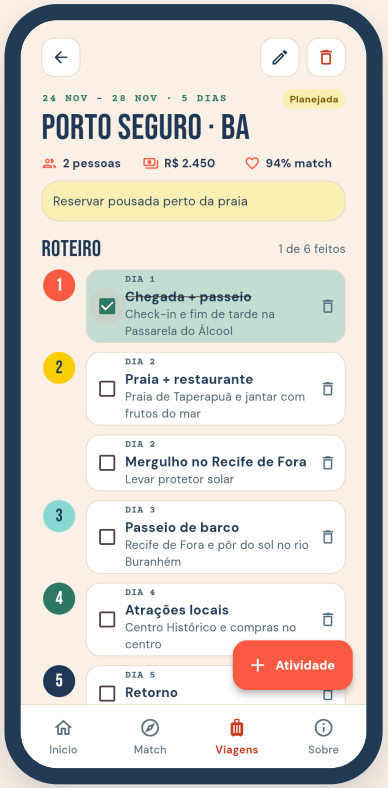
Cada viagem tem seu roteiro na tabela roteiro_itens. Dá para adicionar, editar, marcar como feita e excluir.



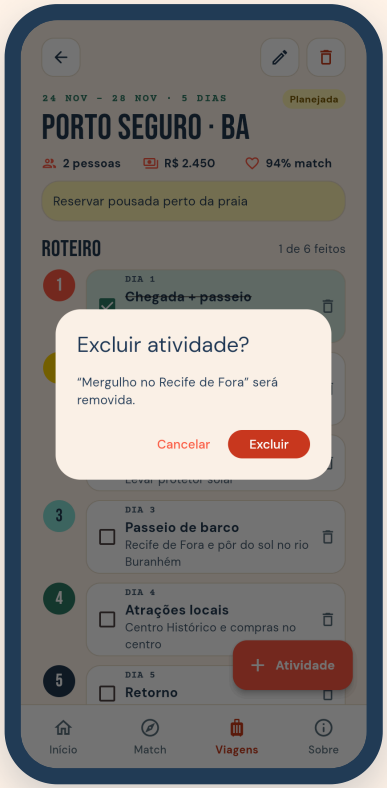
R R · Viagem + roteiro



C C · Nova atividade



U U · Atividade concluída



D D · Excluir atividade

CÓDIGO

COMO O APP CONVERSA COM O BANCO

Trechos de lib/data/. Mesmo padrão do SQLHelper da aula, organizado em uma classe por responsabilidade.

```
// CREATE: viagem + roteiro sugerido (transação)
static Future<int> criar(Viagem v, Destination d) async {
  final db = await DatabaseHelper.db();
  return db.transaction((txn) async {
    final id = await txn.insert('viagens', v.toMap());
    for (final dia in d.itinerary) {
      if (dia.dia > v.dias) break;
      await txn.insert('roteiro_itens', RoteiroItem(
        viagemId: id, dia: dia.dia,
        titulo: dia.titulo, descricao: dia.descricao,
      ).toMap());
    }
    return id;
  });
}

// UPDATE
static Future<int> atualizar(Viagem v) async {
  final db = await DatabaseHelper.db();
  return db.update('viagens', v.toMap(),
      where: 'id = ?', whereArgs: [v.id]);
}

// DELETE (itens do roteiro saem em cascata)
static Future<int> excluir(int id) async {
  final db = await DatabaseHelper.db();
  return db.delete('viagens',
      where: 'id = ?', whereArgs: [id]);
}
```

```
class DatabaseHelper {
  static const nomeArquivo = 'tripwise.db';
  static sql.Database? _db;

  static Future<sql.Database> db() async {
    if (_db != null) return _db!;
    _db = await sql.openDatabase(
      p.join(await sql.getDatabasesPath(), nomeArquivo),
      version: 1,
      onConfigure: (db) =>
          db.execute('PRAGMA foreign_keys = ON'),
      onCreate: _criar,  // CREATE TABLE + dados iniciais
    );
    return _db!;
  }
}
```

```
class Session {   // shared_preferences
  static Future<void> entrar(int usuarioId) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt('usuario_id', usuarioId);
  }
}
```

VALIDAÇÃO

TESTES E ROTEIRO DA DEMONSTRAÇÃO

Os testes rodam o CRUD de verdade em um banco SQLite em memória (test/database_test.dart).

```
$ flutter test
00:02 +0: database_test.dart: banco criado com destinos, custos e estilos
Banco tripwise.db criado (versão 1).
00:03 +1: database_test.dart: usuário: create, read, update e delete
00:03 +2: database_test.dart: travel match ordena destinos de praia…
00:03 +3: database_test.dart: viagem e roteiro: CRUD completo e exclusão em cascata
00:05 +5: All tests passed!

✓ banco criado com destinos, custos e estilos
✓ usuário: create, read, update e delete
✓ travel match ordena destinos de praia dentro do orçamento
✓ viagem e roteiro: CRUD completo e exclusão em cascata
✓ fluxo: splash → início → cadastro (widget_test.dart)
```

1. Abrir o app no emulador Android, criar a conta (C) e mostrar a Home com o nome vindo do banco (R)
2. Fazer o Travel Match, abrir Porto Seguro e favoritar (C em favoritos)
3. “Planejar esta viagem”: escolher as datas e salvar (C em viagens + roteiro)
4. Na viagem: adicionar uma atividade (C), marcar como feita (U) e excluir uma (D)
5. Editar a viagem: mudar o status para Confirmada (U) e conferir em Minhas Viagens (R)
6. Fechar e reabrir o app: continua logado (SharedPreferences) e os dados continuam lá
7. Excluir a viagem (D) e, no Perfil, editar os dados (U) e excluir a conta (D em cascata)